

1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?

- The **State Space** is the set of all possible states reachable from the initial state through valid actions.
- The **Search Tree** is the dynamic structure generated by the algorithm during the agent's offline simulation, created by expanding the successors of already-explored states.

2. (Understand) In Step 1.2, you were instructed to use a `reached` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

- The 'reached set' prevents the agent from getting trapped in infinite loops by converting the algorithm from a standard Tree Search into a Graph Search.
- Without this memory set, a DFS agent would repeatedly revisit the same coordinates and get stuck endlessly tracing the same path in the grid.

3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?

- **BFS** is guaranteed to be optimal (assuming a step cost of 1) because it systematically expands the shallowest unexpanded nodes first, guaranteeing the shortest path is found layer by layer.
- **DFS** expands the deepest unexpanded node first, causing it to blindly plunge down single, continuous paths until it hits a dead end, resulting in erratic, winding, and suboptimal routes.

4. (Evaluate) If we scaled `visual\_grid\_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.

- **BFS** has a space complexity of $O(b^{d+1})$, meaning it must keep every generated node in the frontier queue in memory. On a massive grid, this causes explosive memory growth and crashes.
- **DFS** is much more memory-efficient with a space complexity of $O(bm)$ because it only needs to store a single path (linear in depth) from the root node

down to the current leaf

Once Completed Get a PDF of the completed File and Push into the Week 03 brach in the Github Project

Finalize and Lock Lab 03 Documentation

Incomplete: 0/11 questions answered. Please provide detailed responses for all questions.



FACULTY OF COMPUTING